

Отчет по разработке класса `Quadrilateral` с использованием методики TDD

Предмет: Объектно-ориентированное программирование

Тема задания: Определение класса «Четырехугольник на плоскости»

Методика: Разработка через тестирование (Red-Green-Refactor)

Автор: Чеберук Тимофей, 12 группа

1. Введение

Целью работы является создание класса `Quadrilateral` (Четырехугольник) и вспомогательного класса `Point` (Точка). Согласно заданию, класс должен уметь вычислять площадь, периметр и определять специфический тип четырехугольника (квадрат, прямоугольник, ромб, произвольный). Разработка велась строго по циклу TDD.

2. Ход разработки по методике TDD

Этап 1: "Red" (Написание тестов)

На первом этапе были спроектированы интерфейсы будущих классов и написаны тесты в файле `QuadrilateralTest.java` до реализации самой бизнес-логики.

- **Тесты конструктора:** Проверка на `null`, на вырожденные случаи (три точки на одной линии) и на идентичные точки.
- **Тесты вычислений:** Ожидаемые значения площади и периметра для известных фигур (квадрат со стороной 2, прямоугольник 3x2 и т.д.).
- **Тесты классификации:** Метод `type()` должен возвращать строку ("Square", "Rectangle", "Rhombus", "General") на основе геометрических свойств.
- **Результат:** Тесты не компилируются или падают, так как классы еще не созданы.

Этап 2: "Green" (Реализация минимального кода)

Был написан минимально необходимый код в классе `Quadrilateral`, чтобы тесты начали проходить.

1. **Класс `Point`:** Создан как простой контейнер для координат x и y .
2. **Периметр:** Реализован через суммирование расстояний между соседними точками по формуле:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. **Площадь:** Использована формула Гаусса (формула шнурков) для произвольного многоугольника, что позволяет корректно считать площадь любого выпуклого четырехугольника.
4. **Определение типа:** * Если все стороны равны и диагонали равны — **Квадрат**.
 - a. Если противоположные стороны равны и диагонали равны — **Прямоугольник**.
 - b. Если все стороны равны, но диагонали нет — **Ромб**.
 - c. В остальных случаях — **Произвольный**.

Этап 3: "Refactor" (Рефакторинг и обработка ошибок)

На этом этапе код был оптимизирован, и добавлена строгая валидация.

- **Валидация:** В конструктор добавлена проверка `if (p1 == null || ...)` `throw new IllegalArgumentException()`.
- **Геометрическая проверка:** Добавлена проверка на то, что никакие три точки не лежат на одной прямой, чтобы фигура не превращалась в треугольник или отрезок.
- **Инкапсуляция:** Метод `getVertices()` изменен так, чтобы возвращать копию массива точек, предотвращая изменение состояния объекта извне (защита данных).

3. Описание тестов JUnit

В представленном тестовом классе реализованы:

@ParameterizedTest: Использование CsvSource для массовой проверки различных координат и ожидаемых типов фигур, что исключает дублирование кода тестов.

- **Assetions:** Использование assertEquals с дельтой ($1e-10$) для сравнения чисел с плавающей точкой.

4. Реализация демонстрационного приложения

Помимо тестов, разработано приложение (Main), которое:

1. Создает массив объектов Quadrilateral (включая случайную генерацию).
2. Использует потоки (Stream API) или циклы для группировки объектов по типу.
3. Выводит в консоль:
 - а. Количество фигур каждого типа.
 - б. Фигуру с максимальной и минимальной площадью в каждой группе.

5. Заключение

Применение методики TDD позволило создать надежный класс Quadrilateral с полной проверкой входных данных. Тестовое покрытие гарантирует корректность математических вычислений и стабильную работу приложения при обработке массивов данных.

№	Метод теста	Описание сценария	Ожидаемый результат
1	testConstructorValid	Создание стандартного квадрата с корректными точками	Объект успешно создан (без исключений)

2	testConstructorNullPoints	Передача null вместо одного или нескольких объектов Point	IllegalArgumentException
3	testConstructorDegenerate	Попытка создать фигуру с совпадающими точками (0,0) и (0,0)	IllegalArgumentException
4	testPerimeterSquare	Расчет периметра квадрата со стороной 2	Значение равно 8.0
5	testPerimeterRectangle	Расчет периметра прямоугольника со сторонами 3 и 2	Значение равно 10.0
6	testPerimeterRhombus	Расчет периметра ромба со стороной $4\sqrt{2}$	Значение равно $4\sqrt{2}$ (~5.65)
7	testAreaRectangle	Вычисление площади прямоугольника 3×2	Значение равно 6.0

8	testAreaRhombus	Вычисление площади ромба по заданным координатам	Значение равно 2.0
9	testTypeSquare	Идентификация фигуры с равными сторонами и диагоналями	Возврат строки "Square"
10	testTypeGeneral	Идентификация фигуры без специфических свойств сторон/углов	Возврат строки "General"
11	testGetVertices	Проверка инкапсуляции: изменение полученного массива точек	Оригинальные точки объекта не меняются